Level 3: GROUP BY & HAVING

GROUP BY becomes far more useful once combined with JOINs (sales by region) and date functions (revenue by month/quarter/year). This level also reinforces WHERE-before-GROUP-BY filtering and introduces correlated subqueries for finding the top/bottom row within each group.

Q51. Employees per city EASY

```
SELECT city, COUNT(*) AS emp_count
FROM employees
GROUP BY city
ORDER BY emp_count DESC;
```

WHAT THIS QUERY DOES

A grouped count (like Q31) with an ORDER BY added on top, sorting cities from most employees to fewest — a very common 'top categories' style report.

RESULT ON OUR SAMPLE DATA

city	emp_count
Mumbai	5
Delhi	3
Bangalore	3
Hyderabad	2
Chennai	2

5 rows returned.

Q52. Employees per department EASY

```
SELECT department, COUNT(*) AS emp_count
FROM employees
GROUP BY department;
```

WHAT THIS QUERY DOES

Identical to Q31 — included again here as the cheat sheet's opening example for this GROUP BY-focused section.

RESULT ON OUR SAMPLE DATA

department	emp_count
Finance	3
HR	3

department	emp_count
IT	7
Marketing	2

4 rows returned.

Q53. Salary by designation EASY

```
SELECT designation, AVG(salary) AS avg_salary
FROM employees
GROUP BY designation;
```

WHAT THIS QUERY DOES

Grouping by `designation` instead of `department` — since almost every employee here has a unique job title, most groups end up containing just one person, which is a useful reminder that GROUP BY works fine even when many groups have only a single row.

RESULT ON OUR SAMPLE DATA

designation	avg_salary
Accountant	67,000
Architect	95,000
Developer	72,000
Finance Head	85,000
Financial Analyst	70,000
HR Executive	60,000
HR Manager	65,000
HR Specialist	64,000
Marketing Executive	58,000
Marketing Lead	68,000
Senior Developer	78,500
Solution Architect	88,000
Team Lead	90,000
Technical Manager	91,000

14 rows returned.

Q54. Average salary by city EASY

```
SELECT city, AVG(salary) AS avg_salary
FROM employees
```

```
GROUP BY city
ORDER BY avg_salary DESC;
```

WHAT THIS QUERY DOES

Same grouped-average pattern as Q32/Q53, grouped by `city` this time and sorted so the highest-paying city appears first.

RESULT ON OUR SAMPLE DATA

city	avg_salary
Chennai	79,000
Mumbai	78,600
Bangalore	75,000
Delhi	74,333.33
Hyderabad	65,500

5 rows returned.

Q55. Departments with average salary > 60K MEDIUM

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department
HAVING AVG(salary) > 60000;
```

WHAT THIS QUERY DOES

Another `HAVING` example (following Q35), this time filtering on the group's *average* rather than its *count*. As it happens, every department in this dataset clears 60,000 average salary, so all four groups survive the filter — a good reminder that HAVING doesn't always narrow the result down.

RESULT ON OUR SAMPLE DATA

department	avg_salary
Finance	74,000
HR	63,000
IT	84,714.29
Marketing	63,000

4 rows returned.

Q56. Cities having more than 20 employees MEDIUM

```
SELECT city, COUNT(*) AS emp_count
FROM employees
```

```
GROUP BY city
HAVING COUNT(*) > 20;
```

WHAT THIS QUERY DOES

With only 15 employees total spread across 5 cities, no single city could possibly have more than 20 — so this query correctly returns *zero rows*. That's not a bug or an error: an empty result set is a perfectly valid, meaningful answer ('no city meets this condition'), and real-world queries return this constantly.

RESULT ON OUR SAMPLE DATA

city	emp_count
(no rows returned)	

0 rows returned.

Q57. Count females per department EASY

```
SELECT department, COUNT(*) AS female_count
FROM employees
WHERE gender = 'F'
GROUP BY department;
```

WHAT THIS QUERY DOES

This shows the correct order of operations: `WHERE gender = 'F'` filters down to only female employees *first*, and only *then* does `GROUP BY department` bucket what's left. Filtering before grouping (with WHERE) versus after (with HAVING) is exactly the distinction introduced back in Q35.

RESULT ON OUR SAMPLE DATA

department	female_count
Finance	1
HR	3
IT	1
Marketing	2

4 rows returned.

Q58. Count males per department EASY

```
SELECT department, COUNT(*) AS male_count
FROM employees
WHERE gender = 'M'
GROUP BY department;
```

WHAT THIS QUERY DOES

Mirror image of Q57, filtering for `'M'` instead of `'F'` before grouping. Note that Marketing doesn't appear in this result at all — when a group would end up with zero matching rows, GROUP BY simply omits it rather than showing a row with count 0.

RESULT ON OUR SAMPLE DATA

department	male_count
Finance	2
IT	6

2 rows returned.

Q59. Maximum salary by city EASY

```
SELECT city, MAX(salary) AS max_salary
FROM employees
GROUP BY city;
```

WHAT THIS QUERY DOES

The `MAX()`-per-group pattern from Q33, applied to `city` instead of `department`.

RESULT ON OUR SAMPLE DATA

city	max_salary
Bangalore	95,000
Chennai	88,000
Delhi	90,000
Hyderabad	67,000
Mumbai	91,000

5 rows returned.

Q60. Minimum salary by city EASY

```
SELECT city, MIN(salary) AS min_salary
FROM employees
GROUP BY city;
```

WHAT THIS QUERY DOES

The `MIN()`-per-group counterpart to Q59.

RESULT ON OUR SAMPLE DATA

city	min_salary
Bangalore	58,000
Chennai	70,000
Delhi	65,000
Hyderabad	64,000
Mumbai	60,000

5 rows returned.

Q61. Average experience by department EASY

```
SELECT department, AVG(experience_years) AS avg_experience
FROM employees
GROUP BY department;
```

WHAT THIS QUERY DOES

Same grouped-average recipe as always, this time summarizing `experience_years` instead of salary — useful for questions like 'which department has the most seasoned team.'

RESULT ON OUR SAMPLE DATA

department	avg_experience
Finance	5.33
HR	3.67
IT	6.86
Marketing	2.50

4 rows returned.

Q62. Highest paid employee in every department HARD

```
SELECT department, name, salary
FROM employees e1
WHERE salary = (SELECT MAX(salary) FROM employees e2 WHERE e1.department =
e2.department);
```

WHAT THIS QUERY DOES

This is a **correlated subquery** — notice the inner query references `e1.department`, a column from the *outer* query. That means the subquery can't run just once; it has to re-run separately for every single row of e1, each time asking 'what's the max salary in *this particular row's* department,' and keeping the row only if it matches. It's a different (and more flexible) way to reach the same kind of answer as GROUP BY + MAX, except this version returns the full row — including the

employee's *name* — which plain GROUP BY alone cannot do without extra tricks, since you can't SELECT a non-grouped, non-aggregated column like name directly alongside a GROUP BY.

RESULT ON OUR SAMPLE DATA

department	name	salary
HR	Priya Patel	65,000
Finance	Rahul Singh	85,000
IT	Karthik Nair	95,000
Marketing	Meera Krishnan	68,000

4 rows returned.

Q63. Lowest paid employee in every department HARD

```
SELECT department, name, salary
FROM employees e1
WHERE salary = (SELECT MIN(salary) FROM employees e2 WHERE e1.department =
e2.department);
```

WHAT THIS QUERY DOES

The same correlated-subquery technique as Q62, swapping MAX for MIN to find each department's lowest earner by name instead of its highest.

RESULT ON OUR SAMPLE DATA

department	name	salary
IT	Neha Gupta	72,000
HR	Ananya Menon	60,000
Marketing	Sneha Patel	58,000
Finance	Kavya Nair	67,000

4 rows returned.

Q64. Departments with total salary > 5 lakh MEDIUM

```
SELECT department, SUM(salary) AS total_salary
FROM employees
GROUP BY department
HAVING SUM(salary) > 500000;
```

WHAT THIS QUERY DOES

A third HAVING example, this time filtering on a *summed* total rather than a count or average. Only IT clears half a million in combined salary — the other three

departments, despite having reasonable average salaries, simply don't have enough headcount to push their totals that high.

RESULT ON OUR SAMPLE DATA

department	total_salary
IT	593,000

1 row returned.

Q65. Employees grouped by joining year MEDIUM

```
SELECT YEAR(hire_date) AS join_year, COUNT(*) AS emp_count
FROM employees
GROUP BY YEAR(hire_date)
ORDER BY join_year;
```

WHAT THIS QUERY DOES

YEAR() extracts just the year portion out of a full date (e.g. 2020-01-15 → 2020). This is a very common trick: you can GROUP BY the *result of a function*, not just a plain column, which lets you bucket exact dates into coarser categories like 'year hired.' (This book's practice database runs on SQLite rather than MySQL, so the query actually executed uses SQLite's equivalent date function, `strftime('%Y', hire_date)` — the underlying idea and the result are identical either way.)

RESULT ON OUR SAMPLE DATA

join_year	emp_count
2014	1
2015	1
2016	2
2017	1
2018	1
2019	2
2020	3
2021	2
2022	1
2023	1

10 rows returned.

Q66. Projects by status EASY

```
SELECT status, COUNT(*) AS project_count
FROM projects
GROUP BY status;
```

WHAT THIS QUERY DOES

The familiar grouped-count pattern, this time on the `projects` table — showing how many projects are Active, Completed, or still in Planning.

RESULT ON OUR SAMPLE DATA

status	project_count
Active	3
Completed	2
Planning	1

3 rows returned.

Q67. Orders by month MEDIUM

```
SELECT MONTH(order_date) AS month, COUNT(*) AS order_count
FROM orders
GROUP BY MONTH(order_date)
ORDER BY month;
```

WHAT THIS QUERY DOES

Same 'group by a function of a date column' idea as Q65, using `MONTH()` instead of `YEAR()` to see order volume trends across the calendar. (Executed here using SQLite's `strftime('%m', order_date)`, which returns the month as a zero-padded string like '01', '02'.)

RESULT ON OUR SAMPLE DATA

month	order_count
01	3
02	4
03	4
04	4

4 rows returned.

Q68. Customers by city EASY

```
SELECT city, COUNT(*) AS customer_count
FROM customers
GROUP BY city;
```

WHAT THIS QUERY DOES

Grouped count applied to the `customers` table for the first time — the exact same technique works identically across every table in the database.

RESULT ON OUR SAMPLE DATA

city	customer_count
Bangalore	2
Chennai	1
Delhi	2
Hyderabad	1
Mumbai	3
Pune	1

6 rows returned.

Q69. Products by category EASY

```
SELECT category, COUNT(*) AS product_count
FROM products
GROUP BY category;
```

WHAT THIS QUERY DOES

Grouped count on the `products` table, showing how the product catalog is split across Electronics, Accessories, and Software.

RESULT ON OUR SAMPLE DATA

category	product_count
Accessories	4
Electronics	4
Software	2

3 rows returned.

Q70. Sales by region MEDIUM

```
SELECT c.city AS region, SUM(o.amount) AS total_sales
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY c.city;
```

WHAT THIS QUERY DOES

This is the first query in this book combining a **JOIN** with **GROUP BY** (JOINs are covered fully in Level 4, but this preview shows why they matter here): the `orders` table only has a `customer_id` number, not a city, so we JOIN to `customers` to pull in each order's city, and *then* group and sum by that city. Grouping can happen on a column that came from a joined table just as easily as one from the main table.

RESULT ON OUR SAMPLE DATA

region	total_sales
Bangalore	131,000
Chennai	21,000
Delhi	73,000
Hyderabad	67,000
Mumbai	127,000
Pune	45,000

6 rows returned.

Q71. Revenue by month MEDIUM

```
SELECT MONTH(order_date) AS month, SUM(amount) AS total_revenue
FROM orders
GROUP BY MONTH(order_date)
ORDER BY month;
```

WHAT THIS QUERY DOES

Same technique as Q67, swapping `COUNT(*)` for `SUM(amount)` — instead of 'how many orders per month,' this answers 'how much revenue per month.'

RESULT ON OUR SAMPLE DATA

month	total_revenue
01	50,000
02	145,000
03	150,000
04	119,000

4 rows returned.

Q72. Revenue by quarter MEDIUM

```
SELECT QUARTER(order_date) AS quarter, SUM(amount) AS total_revenue
FROM orders
GROUP BY QUARTER(order_date)
ORDER BY quarter;
```

WHAT THIS QUERY DOES

QUARTER() buckets a date into 1 of 4 three-month periods (Jan-Mar = Q1, Apr-Jun = Q2, and so on) — useful for the kind of quarterly reporting most businesses run on. SQLite has no built-in QUARTER() function, so the version actually executed computes it manually from the month: ((month - 1) / 3) + 1, which produces the exact same grouping.

RESULT ON OUR SAMPLE DATA

quarter	total_revenue
1	345,000
2	119,000

2 rows returned.

Q73. Revenue by year EASY

```
SELECT YEAR(order_date) AS year, SUM(amount) AS total_revenue
FROM orders
GROUP BY YEAR(order_date)
ORDER BY year;
```

WHAT THIS QUERY DOES

Same idea as Q65's YEAR() grouping, now summing revenue instead of counting employees. Since every order in this sample dataset happens to fall in 2024, the result collapses to a single row — with a larger, multi-year dataset this would show one row per year.

RESULT ON OUR SAMPLE DATA

year	total_revenue
2024	464,000

1 row returned.

Q74. Top selling category MEDIUM

```
SELECT p.category, SUM(oi.quantity) AS total_sold
FROM products p
JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.category
ORDER BY total_sold DESC LIMIT 1;
```

WHAT THIS QUERY DOES

Another JOIN + GROUP BY combination: connect products to their order line items to find out how many units of each category were actually sold, then apply the familiar 'sort descending, take 1' pattern to surface the single best-selling category.

RESULT ON OUR SAMPLE DATA

category	total_sold
Electronics	16

1 row returned.

Q75. Bottom selling category MEDIUM

```
SELECT p.category, SUM(oi.quantity) AS total_sold
FROM products p
JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.category
ORDER BY total_sold ASC LIMIT 1;
```

WHAT THIS QUERY DOES

The mirror image of Q74 — sorting ascending instead of descending to find the category with the *fewest* units sold rather than the most.

RESULT ON OUR SAMPLE DATA

category	total_sold
Accessories	2

1 row returned.